

Workshop 3

Introduction

This workshop continues with formal modelling in Alloy and using the Alloy Analyzer. The aim is to familiarise yourself further with Alloy.

Modules, information hiding, and specification

Modularity is a key aspect of good software engineering. Using an interface and information hiding, each module can be used without knowledge of the internal workings.

In specifications, information hiding is less important. This is for two reasons:

1. Due to the abstraction levels used within a specification (e.g. using sets and predicates instead of concrete data types and structured programs), hiding the details is not so important. When we hide a data type or algorithm of a module, we present the interface at a level such that a user can understand what information can be recorded by the module, and what effect the operations can have on this – but not the concrete details (data types and algorithms). These details are presented at a higher level of abstraction, using concepts like sets, functions, and relations, and statements about the assumptions (preconditions) and effects (postconditions).
In a specification, we are already presenting at that abstract level, so hiding this information is not necessary. Further, knowing the data type chosen in a specification is useful (often required!) to understand that specification.
2. Unlike the code of a software system, the reader of a specification is likely to want to understand the details of the entire specification — or at least, most of the details. As such, understanding single modules in isolation are not so useful.

Despite this, it can still be valuable to represent the specification as a coherent set of modules, therefore working towards a design. Therefore, Alloy does have a module concept. This is useful for importing a model from one module into other models.

Your tasks

1. Download and open the `lastpass.als` specification from the LMS.
2. Write an assertion that checks that, after running the `add` operation, no two users have the same password for the same URL unless that was already the case before `add` was run. Check this assertion on your model. Of course, this assertion is trivially false, as users should never know anything the existence of another password. The exercise is intended to have you think about how to model this assertion.
3. Update the `lastpass` model such that the assertion no longer finds a counterexample.
4. Download and open the `cacheMemory.als` file. This file models a cache memory system consisting of main memory and cache memory. Study the model until you are confident that you know what it is doing.
5. What is the meaning of the **one** and **var** keywords in the **sig** declaration?

6. Why does the `read` predicate need the following two lines? Try removing them and see how it affects the model (e.g. whether the `LoadNotObservable` assertion holds).

```
c.main' = c.main  
c.cache' = c.cache
```

7. Write an assertion called `LoadPreserves`, which specifies that when memory is loaded, the main memory does not change. Check this assertion.
8. Write an assertion called `CannotReadInitially`, which specifies that in the initial state, nothing can be read (i.e., the `read` operation is not valid). Check this assertion.
9. Write an operation called `destroy`, which takes a memory address as input and removes that memory address and its associated data, whether the address is in the cache or the main memory.
10. Write an assertion called `OnlyOneMemoryDestroyed`, which specifies that when memory is destroyed by the `destroy` operation, it removes memory from either the cache or main memory, but not both. Does this assertion hold? If not, why not?